

RAG Pipeline - Agentic RAG: Adaptive RAG

1. Definition

Adaptive RAG is a strategy that dynamically adjusts the retrieval approach based on the **complexity** of the user's query.

Unlike standard RAG (which always retrieves) or CRAG (which retrieves then corrects), Adaptive RAG **classifies** the query *first* to determine if retrieval is even necessary, and if so, how much. It moves the decision-making to the very beginning of the pipeline.

2. Core Components

A. Query Analysis (The Classifier)

This is the distinct feature of Adaptive RAG. A smaller, faster LLM (or a router) analyzes the user's input and routes it to one of three strategies:

1. **No Retrieval:** For simple, general knowledge queries (e.g., "Write a python function to print hello"). The LLM answers directly from its training data.
2. **Single-Shot RAG:** For straightforward factual queries (e.g., "What is the refund policy?"). The system performs one standard retrieval and generation step.
3. **Iterative / Agentic RAG:** For complex, multi-hop queries (e.g., "Compare the features of Product A vs. Product B and conclude which is better for enterprise"). The system enters a multi-step loop of retrieval, reasoning, and re-retrieval.

B. Online Testing (Validation Loop)

This refers to the **Self-Correction** mechanism.

- **Concept:** Similar to "Unit Testing" in software, the agent generates an answer and then runs a "test" on it (using an LLM-as-a-Judge).
- **Assertions:**
 - "Is the answer grounded in the facts?" (Faithfulness)
 - "Does the answer address the user's question?" (Relevance)
- **Retry:** If a test fails, the agent automatically "retries"—it might re-write the query, fetch new documents, or re-generate the answer.

3. Specialized Models: Command R

While Adaptive RAG is an *architecture* that can run on any LLM (like GPT-4), specialized models like **Cohere's Command R** are often used to implement it efficiently.

- **Why Command R?**
 - **Tool Use Optimization:** It is fine-tuned to select tools (routes) accurately, which is critical for the "Classifier" step.
 - **Long Context (128k):** It can ingest massive amounts of retrieved documents in the "Complex" path without crashing.
 - **Citations:** It is trained to explicitly cite where it got its information, making the "Validation" step much easier and more reliable.

4. The Pipeline Architecture (Build Order)

Creating an Adaptive RAG system involves assembling the following components into a cohesive graph. This is the order in which you define and build them in code:

1. **Indexer:** The foundation. Sets up the **Vector Store** (for unstructured data), **SQL** (for structured data), or **NoSQL** databases.
2. **Retrieval:** The standard retriever component (using dense, sparse, or hybrid search) to fetch documents from the index.
3. **Router:** The "brain" that classifies the query (Simple vs. Complex) and directs the flow (e.g., to Vector Store vs. Web Search).
4. **Grader:** A node that evaluates the retrieved documents for relevance *before* generation (similar to CRAG).
5. **Generation:** The LLM synthesizes the answer using the context.
6. **Fall Back:** A safety mechanism. If retrieval fails or documents are irrelevant, the system falls back to a secondary method (like Web Search).
7. **Hallucination Grader:** A check run *after* generation. It compares the *answer* vs. the *documents* to ensure the LLM didn't make things up.
8. **Answer Grader:** A final check. It compares the *answer* vs. the *user's question* to ensure the question was actually resolved.
9. **Search Tool:** An external tool (e.g., Tavily, Google) used by the Router or Fall Back mechanisms for "Open Book" questions.
10. **Create a Graph:** The final step is to wrap all these nodes and conditional edges into a **LangGraph** object to manage the state and cyclic transitions.

5. The Execution Flow (Runtime)

When a user actually asks a question, the system executes the components in this dynamic order:

1. **Input:** User submits a query.
2. **Router (Classifier):** The system first analyzes the query complexity.
 - *If Simple:* Jumps straight to **Generation**.

- *If Complex/Data-Dependent*: Proceeds to **Retrieval**.
- 3. **Retrieval (Conditional)**: Fetches documents from the Vector Store or Web Search tool based on the Router's decision.
- 4. **Grader (Pre-Generation)**: Evaluates the retrieved documents.
 - *If Irrelevant*: Triggers **Fall Back** (e.g., Web Search) or Query Rewriting.
 - *If Relevant*: Proceeds to **Generation**.
- 5. **Generation**: The LLM synthesizes an answer.
- 6. **Hallucination/Answer Grader (Post-Generation)**: The system "unit tests" the answer.
 - *If Fail*: The graph loops back to **Retrieval** or **Generation** to retry.
 - *If Pass*: The final answer is delivered to the user.

6. Key LangChain Tools

Component	Tool / Implementation
Routing	LangGraph : Essential for building the branching logic (Simple vs. Complex paths) and the loops.
Classifier	ChatPromptTemplate (Structured Output) : A prompt that forces the LLM to output a category (simple, moderate, complex).
Validation	PydanticOutputParser : Used to enforce a binary pass/fail grade from the evaluator LLM.
Model	ChatCohere (Command R) : Excellent choice for the driving LLM due to high performance-to-cost ratio and RAG capabilities.